SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING	
Program Name:B. Tech		**Assignment Type: Lab**	**Academic Year:**2025-2026
Course Coordinator Name	Dr. Rishabh Mittal		
Instructor(s)Name	Mr. S Naresh Kumar Ms. B. Swathi Dr. Sasanko Shekhar Gantayat Mr. Md Sallauddin Dr. Mathivanan Mr. Y Srikanth Ms. N Shilpa Dr. Rishabh Mittal (Coordinator) Dr. R. Prashant Kumar Mr. Ankushavali MD Mr. B Viswanath Ms. Sujitha Reddy Ms. A. Anitha Ms. M.Madhuri Ms. Katherashala Swetha Ms. Velpula sumalatha Mr. Bingi Raju Mr. G. Kranthi		
Course Code	23CS002PC304	**Course Title**	AI Assisted Coding
Year/Sem	III/I	**Regulation**	R23
Date and Day of Assignment	Week 9 - Thursday	**Time(s)**	23CSBTB01 To 23CSBTB52
Duration	2 Hours	**Applicable to Batches**	All Batches
AssignmentNumber:18.4 (Present assignment number)/**24**(Total number of assignments)			

Q.No.	Question	*ExpectedTime to complete*
1	**Lab 18: API Integration: Connecting to External Services with Error Handling**	Week 9

Lab Objectives

- Learn how to integrate Python programs with external REST APIs.
- Understand API request/response handling using requests or similar libraries.
- Implement proper error handling for failed API calls (timeouts, invalid responses, rate limits).
- Practice extracting and displaying meaningful information from API responses.

Learning Outcome

After completing this assignment, students will be able to:

- Integrate external REST APIs using AI-assisted Python scripting.
- Implement structured error handling for API failures and invalid inputs.
- Safely parse and validate JSON responses from external services.
- Develop resilient applications capable of handling real-world API limitations.

Task 1 – Movie Database API Integration

Scenario

You are building a movie recommendation tool that fetches movie details from a public Movie Database API.

Task

Use AI to generate a Python script that connects to a Movie API and retrieves movie information by title.

Instructions

- Use `requests` library to fetch movie details.
- Accept movie title as user input.
- Display:
  - Title
  - Release year
  - Rating
  - Plot summary
- Implement error handling for:
  - Invalid movie name
  - API key errors
  - Network timeout

- Empty API response

Expected Output

- A working Python script that fetches movie details.
- Graceful error messages instead of crashes.
- Clean, readable output format.

Prompt:-

Create a Python program using the requests library to fetch movie details from an API based on user input. The program should display title, year, rating, and plot, and handle errors like invalid movie name and network issues.

Code & Output:-

```python
import requests

def get_movie(title):
    url = f"http://www.omdbapi.com/?t={title}&apikey=thewdb"

    try:
        response = requests.get(url, timeout=5)
        data = response.json()

        if data.get("Response") == "False":
            print("Error: Movie not found")
            return

        print("\n--- Movie Details ---")
        print("Title:", data.get("Title"))
        print("Year:", data.get("Year"))
        print("Rating:", data.get("imdbRating"))
        print("Plot:", data.get("Plot"))

    except:
        print("Error occurred")

movie = input("Enter movie name: ")
get_movie(movie)
```

```
Enter movie name: made in korea

--- Movie Details ---
Title: Made in Korea
Year: 2026
Rating: N/A
Plot: After her boyfriend's betrayal leaves her alone in Seoul, Shenba faces isolation and cultural differences. Through personal growth and new f
```

Explanation:-

The program takes a movie name from the user and sends a request to the API. The API returns movie details in JSON format, which are then extracted and displayed. Error handling ensures the program works even if the movie is not found or if there is a network issue.

Task 2 – Public Transport Schedule API

Scenario

A city transport app needs to fetch bus/train arrival timings from a transport API.

Task

Use AI assistance to integrate a transport schedule API.

Instructions

- Fetch arrival times for a given station/stop ID.
- Validate user input.
- Handle errors such as:
  - Invalid stop ID
  - Server unavailable
  - Timeout errors
- Display next 3 arrival times in formatted output.
- Implement retry logic (minimum 1 retry attempt).

Expected Output

- Python script showing upcoming arrivals.
- Retry mechanism working properly.
- Proper exception handling using try-except blocks.

Prompt:-

M Develop a Python script to fetch transport arrival timings using an API. The program should display the next three arrivals and include retry logic and error handling for failures.

Code& output:-

```python
import time

def get_arrivals():
    # Simulated API response
    data = {
        "arrivals": ["5 mins", "12 mins", "20 mins", "30 mins"]
    }

    for i in range(2):  # retry logic (simulated)
        try:
            arrivals = data.get("arrivals", [])

            if not arrivals:
                print("No data available")
                return

            print("\nNext 3 arrivals:")
            for i in range(min(3, len(arrivals))):
                print(arrivals[i])

            return

        except Exception:
            print("Retrying...")
            time.sleep(2)

get_arrivals()
```

```
Next 3 arrivals:
5 mins
12 mins
20 mins
```

Explanation:-
The program fetches arrival timings from a mock API and displays the next three arrivals. It includes retry logic to handle temporary failures. If the API does not return data, the program shows an appropriate message.

Task 3 – Stock Market Data API

Scenario

You are developing a finance dashboard that retrieves live stock prices.

Task

Use AI to generate a Python script that fetches stock data from a financial API.

Instructions

- Accept stock symbol as input.
- Retrieve:
    - Current price
    - Day high
    - Day low
- Handle:
    - Invalid stock symbol

- API rate limits
- Missing fields in JSON response

- Validate and safely parse JSON response.

Expected Output

- Script fetches and displays stock information.
- No unhandled exceptions.
- Clear error messages for invalid symbols or rate limits.

Prompt:-

Write a Python program to retrieve stock market data using an API by taking stock symbol as input. Display current price, high, and low values with proper error handling.

Code and output:-

```python
import requests

API_KEY = "BB0TW24A45T0F7FT"

def get_stock(symbol):
    url = f"https://www.alphavantage.co/query?function=GLOBAL_QUOTE&symbol={symbol}&apikey={API_KEY}"

    try:
        response = requests.get(url, timeout=5)
        data = response.json()

        # Check API limit
        if "Note" in data:
            print("API limit reached. Try again later.")
            return

        stock = data.get("Global Quote")

        if not stock:
            print("Invalid stock symbol")
            return

        print("\n--- Stock Details ---")
        print("Price:", stock.get("05. price"))
        print("High:", stock.get("03. high"))
        print("Low:", stock.get("04. low"))

    except requests.exceptions.Timeout:
        print("Timeout error")

    except Exception:
        print("Error fetching stock data")

symbol = input("Enter stock symbol: ")
get_stock(symbol)
```

```
Enter stock symbol: IBM

--- Stock Details ---
Price: 241.3900
High: 246.1900
Low: 238.0000
```

Explanation:-

The user enters a stock symbol and the program sends a request to the API. The API returns stock details such as price, high, and low values. The program safely extracts and displays the data while handling errors like invalid symbols or API limits.

Task 4 – Geolocation API (IP Address Lookup)

Scenario

A cybersecurity tool needs to detect the geographical location of a user based on IP address.

Task

Integrate a Geolocation API using AI assistance.

Instructions

- Accept IP address as input.
- Fetch:
  - Country
  - City
  - ISP
- Validate IP format before making API call.
- Handle:
  - Invalid IP format
  - API failure
  - JSON decoding errors

Expected Output

- A script that returns geolocation details.
- Input validation implemented.
- Robust error handling included.

Prompt:-

M Create a Python script to find location details using an IP address through a geolocation API. Validate input and display country, city, and ISP with error handling.

Code& output:-

```python
def get_location(ip):
    # Check IP format
    if not valid_ip(ip):
        print("Invalid IP format")
        return

    url = f"http://ip-api.com/json/{ip}"

    try:
        response = requests.get(url, timeout=5)
        data = response.json()

        # Check API response
        if data.get("status") != "success":
            print("API failure")
            return

        print("\n--- Location Details ---")
        print("Country:", data.get("country"))
        print("City:", data.get("city"))
        print("ISP:", data.get("isp"))

    except requests.exceptions.Timeout:
        print("Timeout error")

    except Exception:
        print("Error fetching location")

# Input
ip = input("Enter IP address: ")
get_location(ip)
```

```
Enter IP address: 49.37.0.1

--- Location Details ---
Country: India
City: Kolkata
ISP: Reliance Jio Infocomm Limited
```

Explanation:-
The program takes an IP address from the user and validates its format. It then sends a request to the API, which returns location details like country, city, and ISP. The program displays this information and handles errors if the input is invalid or the API fails

Task 5 – Real-Time Application: Payment Gateway Simulation API

Scenario

You are building a test payment integration system that connects to a sandbox payment API.

Task

Use AI to generate a Python script that simulates payment processing.

Instructions

- Send POST request with:
  - Amount
  - Currency

- Payment method
- Validate input values.
- Handle:
  - Invalid payment details
  - HTTP status errors (400, 401, 500)
  - Network interruptions
- Display transaction status clearly.
- Log failed transactions to a file.

Expected Output

- Script processes simulated payment.
- Errors handled gracefully.
- Failed requests logged properly.

Prompt:-

Develop a Python program to simulate a payment system using a POST request. Validate inputs and handle errors such as invalid details and network failures.

Working...

Code& output:-

```python
import requests

def process_payment(amount, currency, method):
    url = "https://httpbin.org/post"

    # Input validation
    if float(amount) <= 0:
        print("Invalid amount")
        return

    if currency not in ["INR", "USD"]:
        print("Invalid currency")
        return

    if method not in ["UPI", "CARD", "NETBANKING"]:
        print("Invalid payment method")
        return

    data = {
        "amount": amount,
        "currency": currency,
        "method": method
    }

    try:
        response = requests.post(url, json=data, timeout=5)

        # Handle HTTP errors
        if response.status_code == 200:
            print("\nPayment Successful ✅")
            print("Transaction Details:", data)
        elif response.status_code == 400:
            print("Bad Request ❌")
        elif response.status_code == 401:
            print("Unauthorized ❌")
        else:
            print("Server Error ❌")

    except requests.exceptions.Timeout:
        print("Timeout error ❌")
```

```
            print("Transaction Details:", data)
        elif response.status_code == 400:
            print("Bad Request ❌")
        elif response.status_code == 401:
            print("Unauthorized ❌")
        else:
            print("Server Error ❌")

    except requests.exceptions.Timeout:
        print("Timeout error ❌")

        with open("failed_log.txt", "a") as f:
            f.write(str(data) + "\n")

    except Exception:
        print("Network error ❌")

        with open("failed_log.txt", "a") as f:
            f.write(str(data) + "\n")


# INPUTS
amount = input("Enter amount: ")
currency = input("Enter currency (INR/USD): ")
method = input("Enter payment method (UPI/CARD/NETBANKING): ")

process_payment(amount, currency, method)
```

```
Enter amount: 500
Enter currency (INR/USD): INR
Enter payment method (UPI/CARD/NETBANKING): UPI

Payment Successful ✅
Transaction Details: {'amount': '500', 'currency': 'INR', 'method': 'UPI'}
```

Explanation:-

The program takes payment details such as amount, currency, and method from the user. It sends this data using a POST request to simulate a payment process. The program displays success or failure and logs errors if the transaction fails.

Note: Report should be submitted a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots